

Dự án Project 1.6 – Tài liệu Mô tả Kỹ thuật

Giới thiệu

Project 1.6 nhằm xây dựng một hệ thống thu thập và phân tích dữ liệu thị trường chứng khoán Hoa Kỳ từ nhiều nguồn khác nhau, bao gồm dữ liệu giá cổ phiếu hàng ngày, tin tức và thông tin doanh nghiệp. Hệ thống sẽ trích xuất dữ liệu từ các API tài chính, lưu trữ vào **data lake** và **data warehouse**, sau đó cung cấp cơ sở cho các phân tích như xu hướng giá cổ phiếu, tâm lý thị trường (sentiment) và hiệu suất nhóm ngành. Tài liệu này mô tả chi tiết các nguồn dữ liệu, kiến trúc lưu trữ, thiết kế **ETL/ELT** và kế hoạch triển khai hệ thống.

Nguồn Dữ Liệu (Data Sources)

Các nguồn dữ liệu của dự án là các **API** do các tổ chức cung cấp dữ liệu tài chính uy tín cung cấp. Cụ thể gồm các API sau:

- **SEC-API (sec-api.io)** – API truy xuất danh sách các công ty hiện đang (hoặc đã từng) niêm yết trên các sàn chứng khoán lớn của Mỹ (NYSE, NASDAQ). (Xem tài liệu: [sec-api.io – List Companies by Exchange](https://sec-api.io/List-Companies-by-Exchange)).
- **Alpha Vantage – Market Status** – API truy xuất danh sách các thị trường chứng khoán trên toàn thế giới và trạng thái hiện tại (mở/cửa, đóng cửa, ngày nghỉ, v.v.). (Xem tài liệu: [Alpha Vantage Market Status](https://alpha-vantage.com/Market-Status)).
- **Alpha Vantage – News & Sentiment** – API truy xuất tin tức thị trường tài chính kèm dữ liệu phân tích cảm xúc (sentiment) từ nhiều nguồn tin hàng đầu thế giới. (Xem tài liệu: [Alpha Vantage News Sentiment](https://alpha-vantage.com/News-Sentiment)).
- **Polygon.io – Daily OHLC** – API truy xuất giá mở cửa, cao nhất, thấp nhất, đóng cửa hàng ngày (OHLC) và khối lượng giao dịch cho toàn bộ cổ phiếu trên thị trường chứng khoán Hoa Kỳ. (Xem tài liệu: [Polygon Aggregates \(Daily OHLC\)](https://polygon.io/Daily-OHLC)).

Chi tiết từng API và dữ liệu trả về

1. sec-api.io – Danh sách Công ty theo Sàn

- **Mục đích:** Cung cấp danh sách tất cả các công ty (bao gồm cả đang niêm yết và đã hủy niêm yết) trên một sàn giao dịch cụ thể. Hỗ trợ các sàn: NYSE, NASDAQ, NYSEMKT, NYSEARCA, OTC,... Đây là dữ liệu nền tảng để biết danh sách mã cổ phiếu và thông tin cơ bản của từng công ty trên thị trường.
- **Tần suất truy xuất: 1 lần mỗi tháng** (vào khoảng 12h trưa ngày cuối tháng) nhằm cập nhật kịp thời nếu có công ty mới lên sàn hoặc công ty bị hủy niêm yết trong tháng. Do danh sách công ty không thay đổi thường xuyên, tần suất này là đủ để đảm bảo dữ liệu cập nhật mà không gọi API quá nhiều.

- **Tham số truy vấn:** Sử dụng phương thức `GET` với endpoint có dạng:

```
https://api.sec-api.io/mapping/exchange/<EXCHANGE>?token=<API_KEY>
```

Trong đó `<EXCHANGE>` là mã sàn (ví dụ: `NYSE` hoặc `NASDAQ`), và `token` là API key cấp bởi sec-api.io. Ví dụ: `GET /mapping/exchange/nasdaq?token=YOUR_API_KEY`. API không yêu cầu tham số nào khác ngoài mã sàn và API key.

- **Dữ liệu trả về:** Định dạng **JSON**. Mỗi phần tử của mảng JSON là một đối tượng công ty với các thuộc tính chi tiết:

- **name:** Tên công ty.
- **ticker:** Mã giao dịch (ticker) của công ty trên sàn.
- **cik:** Mã định danh CIK (Central Index Key) của công ty (theo đăng ký SEC).
- **cusip:** Mã CUSIP của cổ phiếu công ty.
- **exchange:** Sàn giao dịch nơi cổ phiếu đang niêm yết (ví dụ: NYSE, NASDAQ).
- **isDelisted:** Trạng thái niêm yết (giá trị Boolean – `true` nếu cổ phiếu đã bị hủy niêm yết khỏi sàn).
- **category:** Hạng mục chứng khoán của công ty (ví dụ: “Domestic Common Stock” – cổ phiếu phổ thông trong nước).
- **sector:** Ngành (sector) hoạt động của công ty (cấp ngành rộng, ví dụ: Công nghệ, Tài chính...).
- **industry:** Ngành nghề cụ thể của công ty (chi tiết hơn sector, ví dụ: “Banks - Regional” – ngân hàng khu vực).
- **sic:** Mã SIC (Standard Industrial Classification) của công ty.
- **sicSector:** Mô tả ngành cấp 1 theo chuẩn SIC (tương ứng với mã SIC).
- **sicIndustry:** Mô tả ngành cấp 2 theo chuẩn SIC (chi tiết ngành theo mã SIC).
- **famaSector:** Ngành theo phân loại Fama-French (nếu có).
- **famaIndustry:** Ngành chi tiết theo mô hình Fama-French (nếu có).
- **currency:** Đồng tiền giao dịch cổ phiếu (thường là USD đối với thị trường Mỹ).
- **location:** Địa điểm trụ sở chính của công ty (thành phố, bang/quốc gia).
- **id:** Định danh duy nhất cho bản ghi công ty (do sec-api.io sinh, dạng chuỗi hex).

- **Lưu ý giới hạn:** API yêu cầu API Key hợp lệ. Bản miễn phí của sec-api.io có giới hạn về **số lượt gọi và dữ liệu**: mỗi truy vấn trả tối đa ~10.000 bản ghi. Danh sách công ty trên cả NYSE và NASDAQ có tổng cộng khoảng **28.000** công ty (tính cả lịch sử), do đó có thể cần tách làm nhiều lần gọi (theo từng sàn) để lấy đầy đủ dữ liệu. Ngoài ra, sec-api.io cấp một lượng “credits” miễn phí hạn chế mỗi tháng – với tần suất 1 lần/tháng thì nằm trong hạn mức, nhưng cần quản lý token cẩn thận để không vượt quá giới hạn.

2. Alpha Vantage – Market Status (Trạng thái Thị trường)

- **Mục đích:** Cung cấp thông tin về các **thị trường chứng khoán trên thế giới**, bao gồm giờ mở cửa, giờ đóng cửa theo giờ địa phương và trạng thái hiện tại (đang mở, đã đóng, hay đang nghỉ lễ) của mỗi thị trường. Dữ liệu này cho phép xác định thời gian giao dịch của các sàn theo khu vực (để hỗ trợ việc lên lịch thu thập tin tức, giá cổ phiếu đúng thời điểm thị trường mở).
- **Tần suất truy xuất: 1 lần mỗi tháng** (vào 12h trưa ngày cuối tháng). Dữ liệu thị trường (danh sách sàn và giờ giao dịch) rất ít thay đổi, do đó không cần cập nhật hàng ngày. Việc truy vấn

hàng tháng nhằm phòng trường hợp có sự thay đổi (ví dụ: lịch nghỉ lễ mới, sàn mới thêm vào, v.v.).

- **Tham số truy vấn:** Sử dụng API Alpha Vantage với tham số chức năng `function=MARKET_STATUS`. Ví dụ:

```
https://www.alphavantage.co/query?  
function=MARKET_STATUS&apikey=YOUR_API_KEY
```

API này không yêu cầu tham số đầu vào nào khác; nó sẽ trả về dữ liệu cho tất cả các thị trường chính trên thế giới. Bắt buộc phải cung cấp `apikey` của Alpha Vantage (mỗi người dùng được cấp một API key miễn phí với hạn mức nhất định).

- **Dữ liệu trả về:** Định dạng **JSON**, bao gồm danh sách các thị trường/khu vực. Mỗi phần tử chứa:
 - **market_type:** Loại thị trường (ví dụ: `"Stock Market"` cho cổ phiếu, `"Bond Market"` cho trái phiếu, v.v.).
 - **region:** Khu vực địa lý hoặc quốc gia của thị trường (ví dụ: `"United States"`, `"Europe"` ...).
 - **primary_exchanges:** Danh sách các sàn giao dịch chính thuộc thị trường/khu vực đó (ví dụ: đối với Hoa Kỳ có `"NYSE"`, `"NASDAQ"` ...).
 - **local_open:** Thời gian mở cửa **địa phương** của thị trường (định dạng HH:MM, múi giờ địa phương).
 - **local_close:** Thời gian đóng cửa **địa phương** của thị trường.
 - **current_status:** Trạng thái hiện tại của thị trường tại thời điểm gọi API (ví dụ: `"open"` – đang mở, `"closed"` – đã đóng, `"holiday"` – nghỉ).
- **Lưu ý giới hạn:** API này sử dụng chung **API Key Alpha Vantage**, do đó tuân theo hạn mức của Alpha Vantage. Đối với tài khoản miễn phí, giới hạn thông thường là **5 lượt gọi/phút** và **500 lượt gọi/ngày** (lưu ý: Alpha Vantage có cập nhật hạn mức xuống 25 lượt/ngày cho người dùng mới gần đây). Với tần suất 1 lần/tháng, việc sử dụng API này gần như không ảnh hưởng đến hạn mức chung. Tuy nhiên, cần quản lý tổng thể việc sử dụng API Alpha Vantage khi kết hợp với các API khác (như News Sentiment) để không vượt quá giới hạn ngày.

3. Alpha Vantage – News & Sentiment (Tin tức và Cảm xúc thị trường)

- **Mục đích:** Truy xuất các **tin tức mới nhất về thị trường tài chính** kèm theo phân tích **mức độ tích cực/tiêu cực (sentiment)** của từng tin. Nguồn tin tổng hợp từ nhiều hãng tin uy tín (Bloomberg, Reuters, CNBC, Financial Times, v.v.), giúp nắm bắt **tâm lý thị trường** và các sự kiện đang diễn ra ảnh hưởng đến thị trường.
- **Tần suất truy xuất:** **Nhiều lần trong ngày**. Cụ thể, hệ thống sẽ lên lịch gọi API này tại các mốc:
 - **Trước giờ giao dịch** (trước phiên mở cửa thị trường Mỹ) – để nắm tin tức quan trọng có thể ảnh hưởng thị trường trong ngày.
 - **Trong giờ giao dịch** (cập nhật liên tục mỗi vài giờ khi thị trường đang mở) – để bắt kịp tin tức nóng và phản ứng thị trường.

- **Sau giờ giao dịch** (cuối ngày) – tổng hợp các tin tức sau phiên để phục vụ phân tích phiên giao dịch.

Trung bình dự kiến khoảng 3–6 lần gọi mỗi ngày (sáng, trưa, chiều tối), tùy lưu lượng tin.

- **Tham số truy vấn:** Sử dụng API Alpha Vantage với tham số `function=NEWS_SENTIMENT`. Ngoài ra có thể chỉ định:

- **tickers** (tùy chọn): Mã cổ phiếu liên quan cần lấy tin tức. Có thể cung cấp một hoặc nhiều mã (phân tách bằng dấu phẩy) nếu muốn lọc tin theo cổ phiếu cụ thể. Nếu bỏ trống, API sẽ trả về tin tức thị trường chung.

- **topics** (tùy chọn): Chủ đề quan tâm. Alpha Vantage hỗ trợ các chủ đề như `technology`, `energy`, `finance`, `economy`, v.v. Có thể gọi nhiều lần với các chủ đề khác nhau để bao quát các lĩnh vực tin tức chính. Nếu không chỉ định, API thường trả về tin tức đa dạng trên nhiều chủ đề.

- **time_from, time_to** (tùy chọn): Khoảng thời gian lọc tin (nếu cần tin trong khoảng nhất định). Nếu không chỉ định, ngầm định lấy các tin mới nhất.

- **apikey:** API Key Alpha Vantage (bắt buộc).

Ví dụ: `GET https://www.alphavantage.co/query?function=NEWS_SENTIMENT&tickers=MSFT&topics=technology&apikey=YOUR_API_KEY`
sẽ lấy các tin công nghệ liên quan đến hãng Microsoft.

- **Dữ liệu trả về:** Định dạng **JSON**. Một kết quả truy vấn có thể chứa nhiều bài tin tức, mỗi bài có các trường chính:

- **title:** Tiêu đề bài viết tin tức.
- **url:** Đường dẫn đến bài viết (liên kết nguồn gốc để đọc chi tiết).
- **time_published:** Thời gian xuất bản bài viết (theo định dạng chuẩn, thường là UTC).
- **authors:** Danh sách tác giả của bài (nếu có thông tin).
- **summary:** Tóm tắt ngắn gọn nội dung bài viết.
- **source:** Nguồn tin (tên nhà cung cấp tin, ví dụ: "Reuters", "Bloomberg").
- **source_domain:** Tên miền trang web của nguồn tin (ví dụ: "reuters.com").
- **topics:** Danh sách các **chủ đề** liên quan mà bài viết được gắn nhãn. Mỗi mục trong danh sách có:
 - **topic:** Tên chủ đề (ví dụ: `"Blockchain"`, `"Economy - Macro/Overall"`, `"Energy & Transportation"`, v.v.).
 - **relevance_score:** Điểm số ($0 < x \leq 1$) thể hiện mức độ bài viết liên quan đến chủ đề đó. Điểm càng cao nghĩa là nội dung bài viết tập trung nhiều vào chủ đề.
- **overall_sentiment_score:** Điểm số sentiment tổng thể của bài viết (một số thực dao động khoảng -1 đến 1). Alpha Vantage định nghĩa ngưỡng để phân loại: ví dụ $x \leq -0.35$ rất tiêu cực (Bearish), $x \geq 0.35$ rất tích cực (Bullish), các khoảng giữa cho Neutral hoặc hơi tích cực/tiêu cực.
- **overall_sentiment_label:** Nhãn định tính cho sentiment tổng thể (các giá trị có thể là `"Bearish"`, `"Somewhat-Bearish"`, `"Neutral"`, `"Somewhat-Bullish"`, `"Bullish"` tương ứng với mức điểm sentiment ở trên).
- **ticker_sentiment:** Danh sách các cổ phiếu cụ thể được đề cập trong bài viết và cảm xúc liên quan đến từng cổ phiếu đó. Mỗi phần tử gồm:
 - **ticker:** Mã cổ phiếu được nhắc trong bài.
 - **relevance_score:** Mức độ liên quan của bài viết đối với cổ phiếu này (tương tự như trên, $0 < x \leq 1$).
 - **ticker_sentiment_score:** Điểm sentiment của bài viết đối với cổ phiếu đó (cũng trong khoảng -1 đến 1).

- **ticker_sentiment_label**: Nhãn sentiment dành riêng cho cổ phiếu này (ví dụ: bài viết có thể “Bearish” với cổ phiếu A nhưng “Bullish” với cổ phiếu B tùy nội dung).

(Chú thích: Thuật toán sentiment của Alpha Vantage tự động phân tích ngôn ngữ bài viết. Các ngưỡng điểm sentiment có thể tham khảo: ≤ -0.35 : Bearish, -0.35 đến -0.15 : Somewhat-Bearish, -0.15 đến 0.15 : Neutral, 0.15 đến 0.35 : Somewhat-Bullish, ≥ 0.35 : Bullish. Điểm relevance_score cũng nằm trong khoảng 0–1, càng cao nghĩa là nội dung bài tập trung nhiều vào chủ đề hoặc cổ phiếu đó.)

- **Lưu ý giới hạn**: API này có thể trả khá nhiều dữ liệu (nhiều bài viết mỗi lần gọi). Theo thiết kế Alpha Vantage, mỗi lần gọi **News Sentiment** có thể trả tối đa khoảng **50 bài viết** mới nhất. Để thu thập ~100 bài/ngày, cần gọi 2–3 lần/ngày hoặc tăng tần suất. Tuy nhiên, do **giới hạn chung của Alpha Vantage** (5 lần/phút, 500 lần/ngày trên free tier), cần phân bổ hợp lý số lần gọi giữa API tin tức và các API khác. Ưu tiên gọi theo khoảng thời gian cố định (vd mỗi 4 giờ) để không vượt giới hạn. Nếu nhu cầu tăng cao, có thể cân nhắc nâng cấp API Key (premium) để tăng hạn mức gọi.

4. Polygon.io – Dữ liệu OHLC hàng ngày

- **Mục đích**: Cung cấp dữ liệu **giá cổ phiếu hàng ngày** cho tất cả cổ phiếu trên thị trường Mỹ, bao gồm: **Giá mở cửa (Open)**, **Giá cao nhất (High)**, **Giá thấp nhất (Low)**, **Giá đóng cửa (Close)** và **khối lượng giao dịch (Volume)** mỗi ngày. Dữ liệu này cho phép theo dõi biến động giá lịch sử và khối lượng giao dịch của từng cổ phiếu – là cơ sở để phân tích kỹ thuật và xu hướng thị trường.
- **Tần suất truy xuất: Hàng ngày**. Mỗi ngày, sau khi kết thúc phiên giao dịch (tức là qua ngày hôm sau, khoảng **0h đêm**), hệ thống sẽ gọi API Polygon để lấy dữ liệu của **ngày giao dịch vừa kết thúc**. Lý do gọi vào 0h đêm hôm sau là do API ở gói miễn phí thường chỉ cung cấp dữ liệu cuối ngày sau khi ngày đó hoàn tất (không cung cấp giá intraday realtime cho free tier). Lịch thu thập hàng ngày đảm bảo cập nhật đầy đủ từng ngày giao dịch liên tục.
- **Tham số truy vấn**: Polygon cung cấp endpoint *Aggregates* cho phép lấy dữ liệu OHLC theo khoảng thời gian. Cú pháp chung:

```
GET /v2/aggs/ticker/<TICKER>/range/<MULTIPLIER>/<TIMESPAN>/<FROM>/<TO>?
apiKey=<API_KEY>
```

Trong đó:

- **<TICKER>**: Mã cổ phiếu cần lấy dữ liệu (ví dụ: **AAPL**, **MSFT**).
- **<MULTIPLIER>** và **<TIMESPAN>**: Khoảng thời gian lấy mẫu. Với dữ liệu ngày, dùng **1** và **day** (lấy theo từng ngày).
- **<FROM>** và **<TO>**: Ngày bắt đầu và kết thúc (định dạng YYYY-MM-DD). Có thể đặt **<FROM>** = **<TO>** để lấy dữ liệu của một ngày cụ thể, hoặc khoảng dài hơn để lấy nhiều ngày.
- **apiKey**: API Key của Polygon.

Cách sử dụng cho toàn bộ thị trường: Polygon không có endpoint trả về tất cả tickers cùng lúc; do đó cần **lặp qua từng mã cổ phiếu** hoặc sử dụng nhiều kết nối song song. Trong thực tế, có thể tối ưu bằng cách chia theo nhóm mã hoặc dùng các **đặc tả chung** (ví dụ: một số công cụ hỗ trợ lấy toàn bộ dữ

liệu EOD cho nhiều mã) nếu có. Với gói miễn phí, có thể ưu tiên các mã vốn hóa lớn trước, sau đó bổ sung dần (trong giai đoạn đầu thu thập lịch sử).

- **Dữ liệu trả về:** Định dạng **JSON**. Kết quả cho mỗi mã cổ phiếu bao gồm:
- **ticker (T):** Mã cổ phiếu. (Lưu ý: trong JSON thực, trường này có thể ký hiệu là **T**)
- **v:** Khối lượng giao dịch (volume) trong khoảng thời gian (ở đây là trong ngày).
- **o:** Giá mở cửa (opening price).
- **c:** Giá đóng cửa (closing price).
- **h:** Giá cao nhất (highest price) trong ngày.
- **l:** Giá thấp nhất (lowest price) trong ngày.
- **t:** Timestamp (Unix time) của mốc thời gian (có thể dùng để xác định ngày giao dịch).
- **n:** Số lượng giao dịch (ticks) trong khoảng đó (ít quan trọng hơn, có thể không dùng).

Mỗi kết quả trả về có thể chứa một mảng các bản ghi OHLC nếu khoảng **<FROM>** - **<TO>** có nhiều ngày. Trường hợp lấy 1 ngày sẽ có mảng dài 1. Ví dụ, lấy từ 2024-01-01 đến 2024-01-31 cho mã **AAPL** sẽ trả về ~21 bản ghi (tương ứng ~21 phiên giao dịch trong tháng 1/2024).

- **Lưu ý giới hạn:** Polygon free tier giới hạn **tốc độ gọi API ~5 requests/phút và ~100.000 requests/tháng**. Với thị trường Mỹ có vài nghìn mã cổ phiếu đang hoạt động (ước tính ~8.000+ mã trên NYSE+NASDAQ), **không thể lấy toàn bộ mã chỉ trong một lần gọi**. Phương án xử lý:
- Lần đầu thu thập lịch sử: có thể cần hàng ngàn lượt gọi (8k mã, mỗi mã có thể gọi 1 lần cho toàn bộ giai đoạn lịch sử nếu API cho phép nhiều ngày trong một request). Nên **phân bổ theo thời gian** (ví dụ thu thập cuốn chiếu từng nhóm 100 mã một, tránh vượt 5 call/phút). Có thể mất vài ngày đến vài tuần để có toàn bộ dữ liệu lịch sử cho free tier, hoặc cân nhắc nâng cấp gói nếu cần nhanh hơn.
- Thu thập hàng ngày: Sau khi có dữ liệu lịch sử, cập nhật hàng ngày chỉ cần gọi ~8.000 mã * 1 ngày = 8.000 calls. Với 5 calls/phút (~7200 calls/ngày), ngay cả chạy liên tục cũng không đủ quét hết 8k mã trong một ngày ở free tier. Do đó, cách tiếp cận là **ưu tiên mã quan trọng** (ví dụ các mã trong S&P 500) thu thập hàng ngày, các mã nhỏ hơn có thể thu thập với tần suất thấp hơn (2-3 ngày một lần) để nằm trong hạn mức. Một lựa chọn khác là sử dụng nguồn tổng hợp khác hoặc nâng cấp dịch vụ để có dữ liệu toàn thị trường hàng ngày.
- Mỗi request Polygon cũng giới hạn **50.000 điểm dữ liệu**. Tuy nhiên, dữ liệu OHLC theo ngày một năm chỉ ~250 điểm, 10 năm ~2500 điểm – nằm trong giới hạn, nên có thể lấy nhiều năm trong một lần gọi cho một mã.

(Vi hạn chế trên, dự án hiện tập trung vào các cổ phiếu chính. Về lâu dài nếu cần toàn bộ dữ liệu mọi mã, có thể cần giải pháp khác như Polygon premium hoặc nguồn dữ liệu khác.)

Phạm vi Dữ Liệu Thu Thập

- **Thị trường mục tiêu:** Dự án tập trung vào thị trường chứng khoán **Hoa Kỳ**, với các cổ phiếu niêm yết trên sàn **NYSE** và **NASDAQ** (đơn vị tiền tệ: USD). Các thông tin thị trường liên quan (giờ giao dịch, trạng thái mở/đóng) cũng chủ yếu xoay quanh thị trường Mỹ, ngoài ra có tham chiếu đến các khu vực khác qua API Market Status để mở rộng hiểu biết chung.
- **Khoảng thời gian lịch sử:** Bắt đầu thu thập dữ liệu từ **01/01/2024** đến thời điểm hiện tại (2025). Mục tiêu xây dựng kho dữ liệu đủ dài (ít nhất từ đầu 2024) để phân tích xu hướng năm 2024 và cập nhật liên tục. Dữ liệu sẽ được bổ sung hằng ngày/hằng giờ sau mốc ban đầu để luôn duy trì tính thời sự.
- **Loại dữ liệu thu thập:**

- **Dữ liệu giá cổ phiếu (OHLC):** Chuỗi thời gian giá theo ngày cho mỗi cổ phiếu. Bắt đầu từ 2024-01-01 đến nay, và tiếp tục cập nhật hàng ngày. Những ngày thị trường đóng cửa (cuối tuần, lễ) sẽ không có bản ghi.
- **Dữ liệu tin tức tài chính:** Bao gồm các bài báo, bản tin liên quan đến thị trường chứng khoán (tín kinh tế vĩ mô, chính sách, doanh nghiệp, công nghệ ảnh hưởng thị trường...). Thu thập tin tức từ 2024-01-01 đến nay (các tin cũ có thể giới hạn, chủ yếu tập trung tin mới), và cập nhật liên tục theo ngày. Tin tức được thu thập trên nhiều **chủ đề (topics)**: ví dụ như Blockchain, Kinh tế – Chính sách Tài khóa (thuế, chi tiêu chính phủ), Chính sách Tiền tệ (lãi suất, lạm phát), Kinh tế vĩ mô tổng quan, Năng lượng & Vận tải, Tài chính doanh nghiệp, v.v. Điều này đảm bảo bao phủ các lĩnh vực tin ảnh hưởng đến thị trường.
- **Thông tin doanh nghiệp và thị trường:** Danh sách công ty (~28.000 mã cổ phiếu bao gồm lịch sử), thông tin ngành nghề của từng công ty, danh sách sản phẩm dịch vụ và khu vực thị trường (Mỹ, châu Âu, châu Á...). Đây là dữ liệu tham chiếu, phục vụ việc phân loại, nhóm ngành, và sẽ được cập nhật khi có thay đổi (công ty mới IPO, công ty hủy niêm yết, sản phẩm mới, v.v.).

Ước Lượng Dữ Liệu – Đặc trưng 4Vs

Dựa trên phạm vi trên, ta có ước lượng về **khối lượng dữ liệu, tốc độ phát sinh, độ đa dạng và độ tin cậy** như sau:

- **Volume (Khối lượng dữ liệu):**
 - **Dữ liệu giá cổ phiếu (OHLC):** Mỗi ngày giao dịch có khoảng **10.000 dòng** dữ liệu OHLC (giả sử ~10k mã cổ phiếu giao dịch mỗi ngày trên NYSE+NASDAQ). Như vậy, một năm (~252 ngày giao dịch) sinh ra khoảng **2,5 triệu dòng** OHLC. Hiện tại, từ 01/2024 đến giữa 2025 ~ 1,5 năm dữ liệu, tương đương ~3,75 triệu dòng. Mỗi dòng gồm 6 giá trị chính (open, high, low, close, volume, ticker) và các khóa liên kết, dung lượng lưu trữ ước tính vài trăm MB cho dữ liệu thô CSV và có thể tối ưu ~50-100 MB nếu nén.
 - **Dữ liệu tin tức:** Trung bình **100 bài báo/ngày** (có thể cao hơn vào những ngày thị trường biến động mạnh). Một năm (~365 ngày) có khoảng **36.500 bài viết**. Mỗi bài viết bao gồm văn bản (tiêu đề, tóm tắt) nên dung lượng mỗi bài có thể vài KB. Tổng dung lượng một năm tin tức ước tính vài trăm MB (phụ thuộc độ dài summary và số lượng bài).
 - **Danh sách công ty & thông tin thị trường:** ~28.000 công ty (mỗi công ty ~ vài trăm byte dữ liệu JSON). Toàn bộ danh sách khoảng **28k dòng** ~ vài MB. Mỗi tháng có thể thêm/bớt vài chục dòng (công ty mới/hủy) – không đáng kể so với tổng. Thông tin thị trường (market status) chỉ vài chục dòng JSON (mỗi dòng cho một khu vực thị trường).
 - **Tổng quan:** Hệ thống dự kiến xử lý **vài triệu dòng dữ liệu numeric** (giá cổ phiếu) và **khoảng vài chục nghìn mục tin tức** trong một năm hoạt động. Dung lượng lưu trữ thô có thể lên tới hàng trăm MB đến ~1GB mỗi năm, phần lớn do dữ liệu giá cổ phiếu và văn bản tin tức.
- **Velocity (Tốc độ tạo dữ liệu):**
 - Dữ liệu giá cổ phiếu có tính **định kỳ theo ngày**: cứ mỗi ngày giao dịch kết thúc, có ~10k bản ghi mới. Tốc độ này tương đối chậm rãi và cố định (batch daily).
 - Dữ liệu tin tức có tính **liên tục trong ngày**: tin mới có thể xuất hiện bất kỳ lúc nào, đặc biệt quanh các sự kiện quan trọng. Trung bình vài chục tin mỗi ngày, nhưng có thể dồn dập trong một khoảng ngắn khi có sự kiện (ví dụ: khi FED công bố lãi suất, hàng loạt bài viết phân tích xuất hiện). Hệ thống xử lý tin tức ở tốc độ vài giờ một lần (gần real-time nhưng chưa phải streaming từng phút).

- Dữ liệu danh sách công ty/thị trường thay đổi **rất chậm**: chỉ cần kiểm tra mỗi tháng, hoặc thậm chí hàng quý. Tốc độ phát sinh dữ liệu mới trong nhóm này gần như bằng 0 theo ngày, chỉ phát sinh nhỏ giọt vài bản ghi trong tháng.
- Kết luận: Hệ thống phải xử lý **dữ liệu batch hàng ngày** (giá cổ phiếu), **dữ liệu gần realtime hàng giờ** (tin tức), và **dữ liệu chậm hàng tháng** (danh mục công ty). ETL thiết kế đáp ứng đa tốc độ này, trong đó luồng tin tức yêu cầu theo lịch dày đặc nhất.

• Variety (Đa dạng dữ liệu):

- Dữ liệu thu thập rất đa dạng về **định dạng và tính chất**:
 - *Dữ liệu thời gian (time-series)*: Giá cổ phiếu OHLC là dữ liệu chuỗi thời gian có cấu trúc, định dạng số, dễ lưu trong bảng quan hệ hoặc file cột.
 - *Dữ liệu văn bản (textual)*: Tin tức là dữ liệu bán cấu trúc (JSON với trường văn bản như tiêu đề, tóm tắt) và thông tin định tính (sentiment label). Cần xử lý khác so với dữ liệu số, và lưu trữ có thể dưới dạng JSON hoặc bảng quan hệ với các cột text.
 - *Dữ liệu danh mục/từ điển*: Danh sách công ty, bảng thông tin sàn giao dịch, ngành nghề... là dữ liệu bảng tra cứu có cấu trúc quan hệ rõ ràng, thường được lưu trong database.
 - *Dữ liệu đến từ nhiều nguồn khác nhau*: SEC-API (dữ liệu chính thống từ cơ quan quản lý, định dạng JSON); Alpha Vantage (dịch vụ dữ liệu tài chính, JSON); Polygon (dịch vụ thị trường chứng khoán, JSON). Mỗi nguồn có **đặc thù API riêng**, cần tích hợp trong cùng pipeline.
 - Ngoài ra, khi đưa vào Data Warehouse, dữ liệu được chuẩn hóa thành các bảng Fact-Dimension (theo mô hình quan hệ), tức là một dạng biểu diễn khác nữa so với định dạng thô JSON ban đầu.
- Tính đa dạng này đòi hỏi hệ thống linh hoạt: sử dụng kết hợp **cơ sở dữ liệu quan hệ (PostgreSQL)** cho dữ liệu danh mục, **HDFS** cho lưu file JSON/CSV thô, và công cụ **Spark** để xử lý được cả dữ liệu dạng bảng lẫn dạng văn bản trong JSON.

• Veracity (Độ tin cậy, tính xác thực):

- Nguồn dữ liệu dự án sử dụng có **mức độ tin cậy cao**:
 - SEC-API thực chất là dữ liệu từ SEC (Securities and Exchange Commission) – cơ quan quản lý uy tín, đảm bảo tính chính xác về thông tin công ty.
 - Alpha Vantage và Polygon là các nhà cung cấp dữ liệu tài chính chuyên nghiệp; dữ liệu giá cổ phiếu của họ tương ứng sát với thị trường thực (Alpha Vantage sử dụng dữ liệu tổng hợp, Polygon lấy từ các sàn giao dịch). Mặc dù bản miễn phí có thể có độ trễ hoặc giới hạn, nhưng tính đúng đắn của dữ liệu lịch sử và cuối ngày vẫn được đảm bảo.
- Dữ liệu tin tức và phân tích sentiment có **yếu tố chủ quan** hơn: Tin tức được tổng hợp từ nhiều nguồn báo tài chính lớn, do đó thông tin gốc khá đáng tin. Tuy nhiên, **độ chính xác của sentiment** (thuật toán đánh giá tích cực/tiêu cực) có thể không hoàn hảo – đây là kết quả phân tích ngôn ngữ tự nhiên, có thể sai lệch trong một số trường hợp (mỉa mai, ngữ cảnh phức tạp). Dự án chấp nhận mức độ tin cậy này và coi sentiment là một chỉ báo tham khảo chứ không phải số liệu tuyệt đối.
- Về tính toàn vẹn dữ liệu: do dữ liệu từ API có thể gặp lỗi mạng hoặc giới hạn, hệ thống có cơ chế lưu lại lần cuối thành công và sẽ **kiểm tra chéo** (ví dụ: số lượng bản ghi OHLC mỗi ngày có khớp với số mã kỳ vọng không) để phát hiện thiếu sót. Nếu phát hiện thiếu, hệ thống có thể thu thập bù vào lần kế tiếp.

- Tóm lại, **Veracity** của dữ liệu được duy trì tốt nhờ chọn nguồn cung cấp đáng tin cậy và có chiến lược xử lý đảm bảo chất lượng (loại bỏ trùng lặp, kiểm tra thiếu dữ liệu, v.v.). Các kết quả phân tích vì thế sẽ có nền tảng dữ liệu đủ tin cậy để đưa ra insight.

Thiết Kế Lưu Trữ Dữ Liệu Gốc (Source Database)

Dựa trên đặc điểm dữ liệu, hai API **sec-api.io** (danh sách công ty) và **Alpha Vantage Market Status** (thị trường/khu vực) không cần cập nhật thường xuyên đã được lựa chọn lưu vào một **cơ sở dữ liệu quan hệ** làm nguồn dữ liệu nội bộ. Mục tiêu là **giả lập một nguồn Database** bên cạnh các API, đồng thời giảm tần suất gọi API trực tiếp. Cụ thể:

- **CSDL quan hệ (PostgreSQL)**: Được triển khai để lưu trữ dữ liệu từ hai API trên. CSDL này được thiết kế **chuẩn hóa (3NF)** gồm khoảng **5 bảng** chính:
- Bảng **Companies** – Danh sách công ty và thông tin cơ bản: mỗi bản ghi là một công ty (các trường tương tự như dữ liệu trả về từ sec-api: name, ticker, cik, exchange, sector, industry, isDelisted, ...). Bảng này lưu toàn bộ ~28k công ty, có thể có thêm trường timestamp để biết thời điểm cập nhật.
- Bảng **Exchanges** – Danh mục các sàn giao dịch: liệt kê các sàn (NYSE, NASDAQ, ...) và thuộc tính liên quan (có thể bao gồm cả region, market_type, giờ mở/đóng nếu muốn gộp với Market Status). Mỗi sàn có mã riêng (NYSE, NASD...) và thuộc khu vực nào.
- Bảng **Markets/Regions** – Danh sách thị trường/khu vực: ví dụ "United States", "Europe", "Asia", kèm giờ mở/đóng thị trường chuẩn, múi giờ, v.v. (Thông tin từ API Market Status). Bảng này liên kết với bảng Exchanges (mỗi sàn thuộc một thị trường/khu vực).
- Bảng **Sectors** – Danh mục ngành (nếu cần): Liệt kê các sector (ví dụ Technology, Finance, Healthcare, ...) và có thể liên kết tới nhiều công ty.
- Bảng **Industries** – Danh mục ngành nghề chi tiết (industry), liên kết mỗi industry tới một sector cha.
(Chú thích: Một số thiết kế có thể gộp Sector/Industry trực tiếp vào bảng Companies nếu muốn đơn giản. Ở đây tách riêng để tránh lặp lại chuỗi tên ngành nhiều lần và dễ chỉnh sửa phân loại ngành nếu cần.)

- **Mối quan hệ và ràng buộc**: Các bảng trên liên kết với nhau qua **khóa ngoại**:

- Companies.exchange → Exchanges.exchange_code (mỗi công ty thuộc một sàn).
- Exchanges.region → Markets.region_id (mỗi sàn thuộc một thị trường khu vực).
- Companies.sector_id → Sectors.id, Companies.industry_id → Industries.id (nếu áp dụng tách bảng ngành).
- Industries.sector_id → Sectors.id (mỗi ngành chi tiết có một sector cha).
- Các ràng buộc toàn vẹn đảm bảo dữ liệu hợp lệ (ví dụ: exchange của company phải tồn tại trong bảng Exchanges, etc.).

- **Cập nhật dữ liệu**:

- **Bảng Companies**: ban đầu nhập toàn bộ danh sách từ sec-api (có cờ isDelisted). Hàng tháng, chạy quy trình: gọi lại sec-api, so sánh với dữ liệu hiện có để:
 - Thêm bản ghi mới cho công ty chưa có (công ty IPO mới).
 - Cập nhật cờ isDelisted cho công ty không còn trong danh sách (công ty rời sàn).
 - Cập nhật các thuộc tính có thể thay đổi (ví dụ công ty chuyển sàn → thay đổi trường exchange).

- **Bảng Exchanges/Markets:** gần như cố định, nhưng hàng tháng cũng có thể làm mới từ API Market Status để cập nhật nếu có sàn mới hoặc thay đổi giờ giao dịch. (Thực tế rất hiếm thay đổi, nhưng vẫn lên kế hoạch cập nhật đề phòng).
- **Bảng Sector/Industry:** nếu có, có thể xây dựng thủ công từ đầu theo dữ liệu hiện có. Thường các giá trị sector/industry từ sec-api đã là danh mục cố định; có thể tạo bảng từ danh sách unique các sector/industry ban đầu và chỉ cập nhật nếu có giá trị mới xuất hiện (ít khả năng trong ngắn hạn).

• Lợi ích:

- Giảm phụ thuộc API: Chỉ gọi sec-api và market-status 1 lần/tháng, còn lại truy vấn từ database (nhanh hơn, không tốn hạn mức API).
- Tăng tính đa dạng nguồn: Có một nguồn **Database cục bộ** đóng vai trò data source cùng với các API còn lại.
- Chuẩn hóa dữ liệu: Dữ liệu thô (ví dụ tên ngành, tên sàn) được làm sạch, loại bỏ trùng lặp qua thiết kế 3NF.
- Giúp xây dựng **Dimension table** cho Data Warehouse dễ dàng hơn: Bảng Companies trong DB nguồn chính là tiền thân cho Dim_Companies, có thể kết hợp thêm dữ liệu market để làm giàu thông tin.

(Lưu ý: Database này chỉ phục vụ nội bộ dự án, không phải hệ thống giao dịch live. Dung lượng nhỏ và truy vấn không quá nhiều nên PostgreSQL single instance đủ đáp ứng.)

Yêu Cầu Phân Tích Kinh Doanh (Business Requirements)

Từ dữ liệu thu thập được, nhóm dự án đề xuất bốn nhóm bài toán phân tích chính cần giải quyết. Đây cũng là căn cứ để thiết kế các **Data Mart** và cấu trúc Data Warehouse:

1. Phân tích xu hướng giá cổ phiếu – Stock Price Trend Analysis:

2. **Mục tiêu:** Theo dõi và phân tích diễn biến giá của từng cổ phiếu theo thời gian.

3. Nội dung phân tích:

- Xem biến động giá **OHLC hàng ngày** của cổ phiếu sau mỗi phiên giao dịch. Ví dụ: biểu đồ giá theo ngày của cổ phiếu A trong 3 tháng gần nhất.
- Xác định xu hướng giá theo các khung thời gian linh hoạt: ngày, tuần, tháng, quý, năm. Ví dụ: cổ phiếu A trong 1 năm có xu hướng tăng ổn định hay biến động ra sao theo từng quý.
- Phân loại cổ phiếu theo xu hướng: cổ phiếu nào đang **uptrend** (tăng giá liên tục), **downtrend** (giảm liên tục) hoặc **sideway** (dao động nhưng không rõ xu hướng) trong khoảng thời gian nhất định.
- (Mở rộng) Dự báo xu hướng tương lai dựa trên dữ liệu lịch sử và các chỉ báo (có thể ngoài phạm vi dự án 1.6 nhưng là hướng sử dụng dữ liệu).

4. **Dữ liệu cần:** Bảng giá OHLC theo ngày của từng cổ phiếu, có thể kèm chỉ báo tính toán (SMA, EMA, RSI, v.v. nếu cần cho phân tích sâu hơn).

5. Phân tích tâm lý thị trường – Market Sentiment Analysis:

6. **Mục tiêu:** Đo lường và theo dõi “tâm trạng” chung của thị trường chứng khoán thông qua tin tức và các phản ứng của nhà đầu tư.

7. Nội dung phân tích:

- Tổng hợp sentiment từ các nguồn tin: ví dụ, trong một ngày có bao nhiêu tin tức tích cực vs tiêu cực, xu hướng chung (bullish, bearish, neutral) chiếm ưu thế?
- Xác định các **chủ đề hot** đang ảnh hưởng mạnh đến tâm lý thị trường. Ví dụ: chủ đề “Lãi suất tăng” xuất hiện nhiều với sentiment tiêu cực làm thị trường lo ngại.
- Phân tích sentiment theo chủ đề: chủ đề nào có nhiều tin tích cực, chủ đề nào nhiều tin tiêu cực trong tuần/tháng.
- Liên hệ tin tức với thị trường: ví dụ, khi đa số tin tức về ngành Công nghệ là bullish thì nhóm cổ phiếu Công nghệ có tăng giá không? Tin tức tiêu cực về kinh tế vĩ mô có kéo giảm thị trường chung không?

8. **Dữ liệu cần:** Dữ liệu tin tức đã phân loại chủ đề và sentiment (từ API News). Có thể tổng hợp theo ngày (số lượng tin bullish/bearish mỗi ngày), theo chủ đề, và liên kết với dữ liệu giá để xem tác động.

9. Phân tích khối lượng giao dịch – Trading Volume Analysis:

10. **Mục tiêu:** Nghiên cứu hoạt động giao dịch của cổ phiếu thông qua khối lượng, để tìm dấu hiệu bất thường hoặc xu hướng dòng tiền.

11. Nội dung phân tích:

- Theo dõi **khối lượng giao dịch** (volume) của từng cổ phiếu theo thời gian (ngày, tuần, tháng). Ví dụ: cổ phiếu B trung bình 1 triệu cổ phiếu/ngày, đột nhiên có ngày khối lượng 5 triệu thì đáng chú ý.
- Xác định cổ phiếu có khối lượng đột biến: tìm các ngày có volume cao gấp nhiều lần bình thường, hoặc cổ phiếu có khối lượng tăng liên tục trong nhiều phiên. Đây có thể là dấu hiệu của tin nội bộ hoặc sự quan tâm lớn từ thị trường.
- Phân tích mối quan hệ giữa khối lượng và giá: khi khối lượng tăng đột biến, giá có tăng theo (dòng tiền đổ vào mua, thường giá tăng) hay giá giảm (áp lực bán cao)? Tìm sự đồng thuận hoặc phân kỳ giữa giá và volume.
- So sánh volume giữa các cổ phiếu trong cùng ngành: cổ phiếu nào đang thu hút dòng tiền lớn hơn bình thường so với đồng nghiệp, gợi ý sự quan tâm của nhà đầu tư.

12. **Dữ liệu cần:** Dữ liệu **volume giao dịch hàng ngày** (từ API Polygon). Có thể cần tính thêm các thống kê trung bình volume 5 phiên, 10 phiên để so sánh. Kết hợp với dữ liệu giá (close price) để xem tương quan.

13. Phân tích theo nhóm ngành – Sector/Industry Analysis:

14. **Mục tiêu:** Đánh giá hiệu quả và xu hướng của các nhóm cổ phiếu theo ngành, giúp hiểu bức tranh thị trường theo lĩnh vực thay vì từng mã riêng lẻ.

15. Nội dung phân tích:

- **Hiệu suất ngành:** Tính toán mức tăng/giảm bình quân của cổ phiếu trong một ngành (sector) trong khoảng thời gian (ngày/tuần/tháng/quý). Xếp hạng các ngành: ngành nào đang dẫn đầu (tăng trưởng cao nhất), ngành nào tụt hậu.
- **Độ rủi ro và biến động:** Ngành nào biến động mạnh (cổ phiếu trong ngành có biên độ dao động giá lớn), ngành nào ổn định hơn.
- **Dòng tiền theo ngành:** Có thể dùng tổng khối lượng giao dịch hoặc số lượng tin tức liên quan làm proxy. Ngành nào đang thu hút sự chú ý (volume tăng, nhiều tin tức)?
- **Liên hệ sự kiện:** Khi có sự kiện lớn (ví dụ giá dầu tăng mạnh) thì ngành Năng lượng phản ứng ra sao so với ngành khác? Điều này kết hợp phân tích đa biến (tin tức + giá ngành).

- **Tìm ngành tiềm năng:** Dựa trên hiệu suất quá khứ và tin tức, xác định ngành nào có dấu hiệu tiềm năng đầu tư (ví dụ: ngành Công nghệ giảm sâu nhưng tin tức dần tích cực có thể là cơ hội).

16. **Dữ liệu cần:** Thông tin mỗi cổ phiếu thuộc ngành nào (sector/industry từ sec-api), dữ liệu giá và volume của các cổ phiếu. Sẽ cần tổng hợp hoặc tính chỉ số ngành (trung bình/median của các cổ phiếu trong ngành). Dữ liệu tin tức cũng gắn tag ngành qua ticker, có thể đếm số tin theo ngành.

Các yêu cầu trên định hướng trực tiếp thiết kế lược đồ dữ liệu trong Data Warehouse, đảm bảo có đủ **dimension** để nhóm theo thời gian, ngành, chủ đề tin tức; đủ **fact** để chứa số liệu giá, khối lượng, sentiment phục vụ các phân tích định lượng lẫn định tính.

Thiết Kế Data Warehouse (Kho Dữ Liệu)

Dựa vào các **business requirements**, Data Warehouse của dự án được thiết kế theo mô hình **ngôi sao mở rộng (Galaxy Schema)** với nhiều bảng fact chia sẻ chung một số bảng dimension. Việc này cho phép phân tích đa chiều và kết hợp dữ liệu giữa các mảng (giá cổ phiếu, tin tức, công ty) một cách linh hoạt. Dưới đây là cấu trúc các bảng Dimension và Fact chính:

Bảng Dimension (bảng tra cứu thuộc tính)

- **Dim_Companies** – *Dimension Công ty*: Thông tin về công ty và chứng khoán. Kết hợp dữ liệu từ sec-api (công ty) và Market Status (sàn, khu vực) để cung cấp ngữ cảnh đầy đủ cho mỗi mã cổ phiếu. Các thuộc tính chính:
 - **companyId** (PK): Mã định danh duy nhất cho công ty (có thể dùng ticker hoặc một surrogate key số).
 - **name**: Tên đầy đủ của công ty.
 - **ticker**: Mã chứng khoán (ticker) của công ty trên sàn.
 - **isDelisted**: Cờ trạng thái niêm yết (True nếu cổ phiếu này đã bị hủy niêm yết).
 - **category**: Hạng mục cổ phiếu (loại chứng khoán, ví dụ cổ phiếu thường, quỹ ETF, ADR, v.v.).
 - **currency**: Đồng tiền giao dịch của cổ phiếu (đa số là USD cho sàn Mỹ).
 - **location**: Địa điểm trụ sở chính của công ty (ví dụ: "New York, USA").
 - **exchange**: Tên sàn giao dịch mà cổ phiếu niêm yết (ví dụ: NYSE, NASDAQ).
 - **region**: Khu vực thị trường của sàn (ví dụ: "United States").
 - **industry**: Ngành nghề kinh doanh cụ thể của công ty (ví dụ: "Banks - Regional").
 - **sector**: Ngành cấp cao (sector) của công ty (ví dụ: "Financial Services").
 - **sicIndustry**: Ngành chi tiết theo mã SIC (nếu có, thường tương tự industry).
 - **sicSector**: Ngành cấp cao theo mã SIC (thường tương tự sector).

Bảng Dim_Companies cho phép lọc/groupping dữ liệu theo bất kỳ thuộc tính nào của công ty (theo sàn, theo ngành, theo khu vực...). Việc kết hợp thông tin sàn và khu vực vào chung giúp đơn giản hóa việc truy vấn (tránh phải nối thêm bảng sàn khi phân tích theo khu vực).

- **Dim_News** – *Dimension Tin tức*: Thông tin mô tả mỗi bài viết tin tức. Mỗi bản ghi tương ứng một bài tin (có thể tương ứng với URL duy nhất). Các thuộc tính:
 - **newsId** (PK): Mã định danh bài viết (có thể là một hash từ title+time hoặc một số tăng dần).
 - **title**: Tiêu đề bài viết.
 - **url**: URL nguồn của bài viết.
 - **timePublished**: Thời gian xuất bản (có thể lưu dạng TIMESTAMP hoặc tách ngày – liên kết Dim_Time – tùy thiết kế; ở đây để đơn giản, ta liên kết qua Dim_Time riêng).
 - **author**: Tác giả hoặc nhóm tác giả.

- **summary:** Tóm tắt nội dung bài viết. (Có thể lưu truncated nếu quá dài, hoặc lưu riêng file text nếu cần).
- **source:** Nguồn tin (tên hãng tin).
- **source_domain:** Tên miền trang web nguồn.
- **overall_sentiment_score:** Điểm sentiment tổng thể (float).
- **overall_sentiment_label:** Nhãn sentiment tổng thể (text, ví dụ: Neutral, Bullish...).

Bảng *Dim_News* chủ yếu chứa dữ liệu mô tả và định tính. Các phân tích thường nhóm tin theo thời gian (sử dụng *timePublished*), theo nguồn, hoặc lọc theo sentiment... Bảng này sẽ liên kết tới *Fact* để đếm tin, phân tích sentiment ảnh hưởng cổ phiếu.

- **Dim_Topics** – *Dimension Chủ đề*: Danh mục các chủ đề tin tức. Một bài viết có thể có nhiều chủ đề, do đó cần bảng riêng để liệt kê và cho phép quan hệ nhiều-nhiều. Các thuộc tính:
- **topicId** (PK): ID chủ đề (surrogate key hoặc có thể dùng tên chủ đề làm key nếu ít chủ đề).
- **topicName**: Tên chủ đề tin tức (ví dụ: "Blockchain", "Economy - Fiscal Policy", "Energy & Transportation", v.v.).

Bảng *Dim_Topics* cho phép ta chuẩn hóa danh sách chủ đề, tránh lặp lại chuỗi tên chủ đề nhiều lần. Hơn nữa, có thể nhóm các chủ đề cùng loại hoặc lọc báo cáo tin tức theo một chủ đề cụ thể dễ dàng.

- **Dim_Times** – *Dimension Thời gian*: Bảng thời gian cho phép phân tích dữ liệu theo ngày/tháng/năm dễ dàng. Mặc dù ta có thể dùng trực tiếp kiểu ngày, nhưng bảng *Dim_Times* giúp bổ sung thuộc tính ngày trong tuần, quý... Các thuộc tính:
- **timeId** (PK): ID thời gian (mỗi ngày hoặc mỗi thời điểm duy nhất có một ID; thường dùng ngày làm đơn vị chính, nên *timeId* có thể chính là YYYYMMDD).
- **date**: Ngày (dạng DATE, 2024-05-01...).
- **day**: Ngày trong tháng (1–31).
- **month**: Tháng (1–12).
- **quarter**: Quý (1–4).
- **year**: Năm (yyyy).
- **dayOfWeek**: Thứ trong tuần (Monday–Sunday, có thể lưu text hoặc 1–7).
- **isWeekend**: Có phải cuối tuần không (Boolean),...
- (có thể thêm các cột khác như tên tháng, tên quý nếu cần báo cáo dạng text).

Dim_Times sẽ được liên kết tới các *Fact* thông qua *timeId* để hỗ trợ cuốn lịch phân tích. Lưu ý: Đối với tin tức, *timePublished* chính xác tới giờ, nên ta có thể quyết định làm tròn ngày hoặc tạo *timeId* chi tiết tới từng giờ. Đơn giản ở đây ta chỉ xét theo ngày đăng tin (coi mọi tin cùng ngày có chung *timeId*).

Bảng Fact (bảng sự kiện, chứa số liệu)

- **Fact_Candles** – *Fact Giá cổ phiếu (OHLC)*: Đây là bảng fact chính lưu trữ **giá và khối lượng giao dịch hàng ngày** cho từng cổ phiếu. Mỗi bản ghi tương ứng với **một mã cổ phiếu trong một ngày giao dịch**. Các khóa và cột:
- **companyId** (FK → *Dim_Companies*): Xác định cổ phiếu nào (thuộc công ty nào).
- **timeId** (FK → *Dim_Times*): Xác định ngày giao dịch. Kết hợp *companyId* + *timeId* sẽ xác định duy nhất một bản ghi. (Có thể dùng cột surrogate key *candleId* làm PK, nhưng cũng có thể dùng (*companyId*, *timeId*) làm PK phức hợp nếu không trùng).
- **open**: Giá mở cửa trong phiên.
- **high**: Giá cao nhất trong phiên.
- **low**: Giá thấp nhất trong phiên.
- **close**: Giá đóng cửa phiên.
- **volume**: Khối lượng giao dịch trong ngày.

- **adj_close** (tùy chọn): Giá đóng cửa đã điều chỉnh (nếu có chia cổ tức, tách cổ phiếu, etc. – API Polygon có thể trả sẵn hoặc ta tính, không bắt buộc nhưng hữu ích cho phân tích dài hạn).

Fact_Candles cho phép thực hiện các truy vấn như: lấy chuỗi giá của một cổ phiếu qua time, tính toán tăng giảm (%change) giữa các ngày, tổng hợp giá trung bình của nhóm cổ phiếu theo time (phục vụ phân tích ngành), v.v. Bảng này có thể rất lớn (vài triệu bản ghi) nhưng truy vấn vẫn nhanh nếu được đánh index theo *companyId* và *timeId*, và DuckDB có hiệu năng tốt với dữ liệu cột.

- **Fact_News_Topics** – *Fact Liên kết Tin và Chủ đề*: Bảng mapping cho quan hệ nhiều-nhiều giữa **tin tức** và **chủ đề**. Mỗi bản ghi thể hiện một bài viết đề cập một chủ đề nào đó:
- **newsId** (FK → Dim_News): Bài viết.
- **topicId** (FK → Dim_Topics): Chủ đề.
- **relevance_score**: Điểm liên quan của bài viết đối với chủ đề này (do API cung cấp, giá trị 0–1). (Khóa chính của bảng này thường là cặp (*newsId*, *topicId*) hoặc một surrogate key riêng như *newsTopicId* nếu cần).

Fact_News_Topics cho phép tính toán số lượng tin theo chủ đề, hoặc lọc các tin rất liên quan (score cao) đến một chủ đề cụ thể. Ví dụ: đếm có bao nhiêu bài về chủ đề “Blockchain” trong tháng 6, hoặc lấy danh sách bài viết liên quan mạnh đến “Energy” với score > 0.8...

- **Fact_News_Companies** – *Fact Liên kết Tin và Công ty*: Bảng mapping nhiều-nhiều giữa **tin tức** và **công ty (cổ phiếu)**. Một bài viết có thể nhắc đến nhiều công ty, và một công ty xuất hiện trong nhiều bài viết. Mỗi bản ghi gồm:
- **newsId** (FK → Dim_News).
- **companyId** (FK → Dim_Companies).
- **relevance_score**: Điểm liên quan của bài viết tới công ty (0–1, do API cung cấp trong *ticker_sentiment*).
- **ticker_sentiment_score**: Điểm sentiment của bài viết đối với cổ phiếu công ty đó (ví dụ bài viết có giọng điệu rất tích cực về công ty A sẽ có điểm cao, tiêu cực thì điểm thấp âm).
- **ticker_sentiment_label**: Nhãn sentiment tương ứng (Bullish, Bearish, Neutral... nhưng cụ thể cho công ty này trong ngữ cảnh bài viết). (Tương tự, khóa chính có thể là (*newsId*, *companyId*) hoặc một surrogate key riêng).

Fact_News_Companies cho phép phân tích giao thoa giữa tin tức và cổ phiếu: ví dụ truy vấn “liệt kê các bài báo Bearish về công ty Tesla tuần qua”, hoặc “tìm tất cả cổ phiếu có >=5 tin tức Bullish trong tháng và xem hiệu suất giá của chúng”. Bảng này rất quan trọng để kết nối thế giới thông tin định tính (tin tức) với dữ liệu định lượng (giá cổ phiếu trong *Fact_Candles*).

- (Các Fact bổ sung khác): Nếu cần, có thể tạo **Fact_MarketStatus** cho trạng thái thị trường mỗi ngày (open/close) nhưng thông tin này có thể quản lý trong *Dim_Times* (dùng cho phân tích ví dụ bao nhiêu ngày thị trường mở trong tháng...). Hiện tại không có bảng Fact nào khác, 3 Fact trên đã đủ cho scope yêu cầu.

Mô hình dữ liệu (Galaxy Schema): Với thiết kế trên, **Dim_Companies** và **Dim_Times** là hai dimension lớn dùng chung bởi cả *Fact_Candles* (giá cổ phiếu) và *Fact_News_Companies* (tin tức liên quan cổ phiếu). **Dim_News** kết nối tới hai fact liên quan tin (News_Topics, News_Companies). **Dim_Topics** kết nối *Fact_News_Topics*. Sơ đồ liên kết tạo thành một **constellation** cho phép các câu hỏi phức tạp, ví dụ: “trong khoảng thời gian thị trường tăng mạnh, có chủ đề tin tức nào thường xuyên xuất hiện với sentiment tích cực?”.

(Lược đồ trên được tối ưu hóa theo yêu cầu truy vấn: ví dụ chỉ tạo một bảng *Dim_Companies* thay vì tách công ty và sàn/sector thành nhiều dim, nhằm giảm số phép join khi phân tích, theo mục tiêu “Chỉ 1 bảng *Dim Companies* để tối ưu thời gian truy vấn”).

Thiết Kế Quy Trình ETL/ELT

Để xây dựng được kho dữ liệu như trên, nhóm dự án thiết kế một quy trình **ETL** (Extract – Transform – Load) tự động, vận hành hàng ngày và hàng tháng. Quy trình ETL gồm các thành phần chính sau:

1. Extract – Trích xuất dữ liệu từ nguồn:

2. Viết các **module Python** (hoặc sử dụng các operator có sẵn) để gọi từng API theo lịch định sẵn.

Mỗi nguồn dữ liệu có một module extract riêng:

- Module **Extract SEC-API Companies**: Gọi API sec-api, lấy JSON danh sách công ty theo từng sàn (NYSE, NASDAQ). Chuyển đổi nhẹ kết quả (parse JSON) rồi **ghi vào bảng tạm trong PostgreSQL** (hoặc file). Chạy hàng tháng.
- Module **Extract Market Status**: Gọi API Alpha Vantage (MARKET_STATUS), parse JSON lấy danh sách thị trường. Ghi vào bảng tạm (hoặc file JSON trong data lake). Chạy hàng tháng.
- Module **Extract News**: Gọi API Alpha Vantage (NEWS_SENTIMENT). Để đảm bảo đủ tin, module có thể chạy nhiều lần trong ngày với tham số khác nhau (ví dụ lần 1 không filter – lấy tin chung, lần 2 filter theo một nhóm chủ đề khác). Mỗi lần gọi nhận JSON gồm nhiều bài viết. Module sẽ lưu các bài viết này tạm vào **file JSON hoặc CSV** trong data lake (HDFS) kèm timestamp. Chạy theo lịch nhiều lần/ngày.
- Module **Extract Polygon OHLC**: Gọi API Polygon để lấy giá cổ phiếu. Do số lượng cổ phiếu lớn, module có thể có một danh sách các tickers mục tiêu. Chẳng hạn, ban đầu load danh sách S&P 500. Mỗi lần chạy, gọi API cho từng ticker (hoặc theo nhóm) để lấy dữ liệu của ngày hôm qua. Kết quả được lưu tạm vào **file CSV/Parquet** trên HDFS. Chạy hàng ngày (đêm).

3. Tất cả các module trên đều **kết nối internet tới API**, sử dụng API Key tương ứng. Cần xử lý các trường hợp lỗi (hết hạn mức, lỗi mạng) bằng cách retry hoặc chờ sang lần chạy sau.

4. Load (Stage) – Tải dữ liệu thô về lưu trữ tạm:

5. Dữ liệu từ bước Extract sẽ được lưu vào nơi lưu trữ trung gian:

- Đối với **sec-api và Market Status**: do chọn PostgreSQL làm nơi chứa chính thức, module Extract có thể trực tiếp **UPSERT** dữ liệu vào bảng PostgreSQL. Ví dụ: chèn các công ty mới, cập nhật cờ isDelisted trong bảng Companies. Tương tự cho bảng Exchanges/Markets. (Trường hợp cẩn thận hơn, có thể lưu thô ra file CSV rồi dùng COPY import, nhưng volume nhỏ nên insert trực tiếp cũng ổn).
- Đối với **News và OHLC**: khối lượng dữ liệu lớn và thường xuyên, ta lưu thẳng vào **HDFS** (Data Lake). Cụ thể:
 - Tin tức: Lưu file JSON hoặc CSV theo từng lần chạy. Có thể tổ chức thư mục theo ngày/giờ (vd: `news/2025/06/26/news_10AM.json` chứa tin lúc 10h). Việc này giúp dễ quản lý và có thể đọc gộp sau.
 - OHLC: Lưu file (CSV/parquet) theo ngày. Ví dụ: `ohlcv/2025/06/25/ohlcv_2025-06-25.csv` chứa giá toàn bộ tickers lấy được cho ngày 25/06/2025. Nếu tickers chia nhóm, có thể có nhiều file cho cùng ngày, cần hợp lại khi dùng.

6. Tóm lại, bước Load thô này đảm bảo dữ liệu từ API đã được **thu thập đầy đủ và lưu an toàn** (có lịch sử), trước khi chuyển sang bước transform.

7. Transform – Biến đổi và tích hợp dữ liệu:

8. Sử dụng **Apache Spark** (hoặc các công cụ xử lý batch mạnh) để xử lý dữ liệu thô từ data lake và database, biến thành dạng bảng Fact/Dim sẵn sàng nạp vào Data Warehouse.

9. Các tác vụ transform chính:

- **Transform Companies & Markets:** Đọc dữ liệu từ PostgreSQL (bảng Companies, Exchanges, Markets). Thực hiện join/merge cần thiết để tạo **Dim_Companies**:
- Join Companies với Exchanges để lấy tên sàn, rồi join với Markets để có region cho mỗi công ty.
- Chọn và định dạng các cột đúng như thiết kế Dim_Companies.
- Kết quả: một dataframe hoàn chỉnh cho Dim_Companies (28k bản ghi). Có thể thêm cột companyId (surrogate key) nếu cần (nếu dùng ticker làm key thì không cần surrogate).
- **Transform News:** Đọc tất cả các file JSON tin tức mới trong ngày từ HDFS, cộng với có thể giữ một **bản lưu lịch sử** (dự án có thể thiết kế incremental). Xử lý:
- Chuẩn hóa các bài viết: loại bỏ trùng (nếu cùng URL xuất hiện nhiều lần), bổ sung ID.
- Tách **topics**: Mỗi bài viết có thể có nhiều topic, tạo các bản ghi (newsId, topic) – đây chính là Fact_News_Topics. Tương tự, tách **ticker_sentiment**: tạo các bản ghi (newsId, ticker, relevance, sentiment_score, sentiment_label) – tương ứng Fact_News_Companies, nhưng ở đây ticker là mã, cần chuyển thành companyId.
- Để chuyển ticker → companyId: sử dụng bảng Dim_Companies (hoặc data từ Companies DB) để map. Spark có thể broadcast bảng công ty (28k hàng) và join với dataframe ticker_sentiment để gán companyId cho mỗi ticker. Những ticker nào không tìm thấy trong Dim_Companies (ví dụ cổ phiếu không thuộc NYSE/NASDAQ) có thể bỏ qua hoặc gán companyId null.
- Kết quả: một dataframe cho **Dim_News** (các cột newsId, title, timePublished..., sentiment info) – có thể lưu/ghi đè; một dataframe cho **Fact_News_Topics**; một dataframe cho **Fact_News_Companies**.
- **Transform OHLC:** Đọc file CSV OHLC của ngày mới từ HDFS (và có thể union với dữ liệu cũ nếu cần làm đầy đủ). Xử lý:
- Gắn **companyId** cho mỗi dòng OHLC: mỗi bản ghi OHLC có ticker, join với Dim_Companies (giống trên) để lấy companyId. Bỏ những ticker không có (có thể do ticker OTC hoặc sàn khác nếu lỡ thu thập).
- Gắn **timeId**: trích date từ tên file hoặc từ timestamp trong dữ liệu (nếu polygon cung cấp) để biết thuộc ngày nào. Tạo hoặc tra cứu Dim_Times để đổi thành timeId.
- Chuẩn hóa các cột open, close,... đúng kiểu dữ liệu (FLOAT).
- Kết quả: dataframe Fact_Candles mới của ngày. Có thể hợp với fact lịch sử để tạo đầy đủ Fact_Candles, hoặc insert incrementally từng ngày vào warehouse.

10. Bước transform trên Spark sẽ xuất ra dữ liệu đã sẵn sàng cho Data Warehouse. Lợi thế dùng Spark là khả năng xử lý phân tán, tuy nhiên với dữ liệu hiện tại (vài triệu dòng) thì một máy cũng xử lý được, nhưng dùng Spark chuẩn bị cho mở rộng sau này.

11. Load – Nạp vào Data Warehouse:

12. **DuckDB** được sử dụng làm Data Warehouse. DuckDB có thể hoạt động embed trong Python hoặc standalone. Ở đây triển khai theo dạng chạy câu lệnh SQL DuckDB trong môi trường Python/Spark để tạo bảng và chèn dữ liệu.

13. Cách nạp dữ liệu:

- Tạo các bảng Dim và Fact trong DuckDB (lần đầu). Định nghĩa schema đúng thiết kế (các cột và kiểu dữ liệu).

- Nạp **Dim_Companies**: vì số lượng không quá lớn, có thể nạp bằng cách xuất dataframe Spark thành file Parquet rồi dùng DuckDB đọc Parquet trực tiếp vào bảng. Hoặc dùng lệnh INSERT nhiều giá trị (DuckDB khá nhanh với insert bulk). Dim_Companies cập nhật hàng tháng (có thể MERGE để thay đổi).
 - Nạp **Dim_Topics**: trích unique topic từ Fact_News_Topics rồi insert (chủ đề mới nếu có). Thường chủ đề không đổi nhiều.
 - Nạp **Dim_News**: insert các tin mới. Mỗi lần ETL sẽ thêm những newsId chưa có. DuckDB có thể upsert nhưng nếu không hỗ trợ, ta kiểm soát unique bằng cách không insert trùng khóa.
 - Nạp **Dim_Times**: tương tự, thêm những ngày mới nếu chưa có (thường mỗi ngày chạy thêm 1 ngày).
 - Nạp **Fact_Candles**: thêm dữ liệu ngày mới (append vào bảng Fact_Candles). Bảng này rất lớn nhưng append theo ngày khá thuận tiện.
 - Nạp **Fact_News_Topics** và **Fact_News_Companies**: append các bản ghi mới từ tin mới.
14. Vì DuckDB có thể truy cập file hệ thống, một chiến lược hiệu quả là **lưu các Fact, Dim dưới dạng file Parquet** trên disk/HDFS, sau đó tạo **view bên DuckDB** đọc trực tiếp các file đó. Tuy nhiên, để đơn giản người dùng có thể copy dữ liệu vào DuckDB trực tiếp để có một file .duckdb duy nhất làm warehouse.
15. **Orchestration (Điều phối)**:
16. Sử dụng **Apache Airflow** để quản lý toàn bộ các bước trên. Một DAG (Directed Acyclic Graph) ETL sẽ được tạo cho luồng hàng ngày và hàng tháng:
- **DAG hàng ngày** (ví dụ đặt tên `daily_etl`): Gồm các task tuần tự: Extract News (có thể gồm sub-task cho mỗi lần gọi API tin), Extract OHLC, Load thô (hdfs, file), Transform News, Transform OHLC, Load Warehouse (append Fact). Lên lịch chạy **hàng ngày lúc 00:30** (sau khi thị trường đóng và data có sẵn).
 - **DAG hàng tháng** (`monthly_etl`): Chạy vào ngày cuối tháng 12:30, gồm các task: Extract sec-api, Extract market status, Load vào PostgreSQL, Transform Companies/Markets, Load Dim_Companies (update DW). Có thể kèm task làm dọn dẹp (xóa file log cũ, nén dữ liệu...).
17. Airflow **scheduler** tự động chạy DAG theo lịch, **worker** thực thi các task (có thể là LocalExecutor hoặc CeleryExecutor tùy triển khai). Các kết nối (Connection) trong Airflow được cấu hình để kết nối đến PostgreSQL, HDFS, etc., giúp tasks dễ tương tác.
18. Airflow cũng giám sát, gửi **cảnh báo** nếu task thất bại. Ví dụ: nếu Extract News fail do giới hạn API, Airflow có thể retry sau 15 phút (cấu hình retries). Nhờ đó, quy trình ETL có độ tin cậy cao và cần ít sự can thiệp thủ công.

(Ngoài Airflow, có thể sử dụng cron + script, nhưng Airflow cung cấp UI trực quan theo dõi pipeline nên đã được lựa chọn.)

Triển Khai Hệ Thống (Deployment)

Để đảm bảo sự nhất quán và dễ dàng trong việc cài đặt môi trường phức tạp (nhiều thành phần), dự án sử dụng công nghệ **container hóa với Docker**, cụ thể là dùng **Docker Compose** để triển khai toàn bộ hệ thống trên một hoặc nhiều máy chủ. Cấu trúc deployment như sau:

- **Docker Compose**: Một tệp `docker-compose.yml` được viết để định nghĩa các **service container** tương ứng với mỗi thành phần của hệ thống. Các service chính bao gồm:

- **PostgreSQL Service:** Sử dụng image postgres chính thức. Container này chạy PostgreSQL server, khởi tạo các database/tables cần thiết (có thể dùng volume mount script init). Dữ liệu (tables Companies, Exchanges...) được lưu trên một Docker volume để đảm bảo dữ liệu không mất khi container restart. Cấu hình expose cổng (5432) và user/password trong compose.
- **HDFS (Hadoop) Service:** Có thể dùng một image Hadoop single-node (ví dụ: `bde1/hadoop-single-node` giả định) chạy namenode và datanode trong cùng container. Service này cung cấp HDFS trên cổng 9000 (hoặc theo image) và một volume mount để lưu data hdfs (nếu cần). YARN (ResourceManager, NodeManager) cũng có thể chạy trong container này để hỗ trợ Spark.
- **Spark Service:** Triển khai Spark (ví dụ image `bitnami/spark` hoặc custom). Spark có thể chạy ở mode standalone master + worker trong cùng container (cho đơn giản) hoặc tách service Master/Worker. Nhưng với khối lượng nhỏ, một container Spark standalone là đủ. Spark container được cấu hình để biết địa chỉ HDFS (thông qua `CORE_CONF_fs_defaultFS` env) và có network để kết nối các container khác.
- **Airflow Service:** Sử dụng image chính thức của Airflow (ví dụ: `apache/airflow:2.5.0`). Trong compose có thể tách thành 2 services: `airflow-webserver` và `airflow-scheduler` (cộng thêm `airflow-postgres` và `airflow-redis` nếu dùng Celery executor). Để đơn giản, có thể dùng LocalExecutor nên chỉ cần một container cho webserver & scheduler cùng chạy. Airflow container cấu hình volume để chứa DAG files (mount thư mục DAG code từ host), và environment để kết nối đến các service khác (Postgres, etc.) – ví dụ thiết lập `AIRFLOW_CONN_POSTGRES_DEFAULT` cho chuỗi kết nối Postgres, v.v.
- **DuckDB Service:** DuckDB không có server standalone, nhưng ta có thể dùng một container (ví dụ python:3.10) để chạy một lightweight server hoặc cung cấp giao diện query DuckDB. Thực tế, có thể **tích hợp DuckDB vào container Spark hoặc Airflow** vì chúng có sẵn Python environment. Phương án: cài DuckDB-Python trong container Airflow để Airflow task có thể trực tiếp tương tác file DuckDB. Tuy nhiên, nếu cần tách, ta có container DuckDB chạy một script Python nhỏ chờ query (ít phổ biến). Trong triển khai này, ta chọn **không tách riêng container** cho DuckDB; file warehouse `.duckdb` sẽ được lưu trên volume và truy cập bởi Airflow/Spark container.
- **ETL Modules Service:** Các module ETL (extract, transform code) thực ra sẽ chạy bên trong context Airflow (như PythonOperator hoặc BashOperator). Do đó không cần container riêng cho chúng. Tuy nhiên, nếu muốn tách, có thể làm container Python riêng chứa mã ETL và dùng Airflow Triggerer điều khiển. Đơn giản hơn, code ETL được đóng gói trong Docker image Airflow (cài các thư viện `requests`, `pyspark`, `duckdb` vào Airflow image), do đó Airflow vừa điều phối vừa thực thi luôn.

• Mạng và Volume:

- Docker Compose sẽ tạo một **network chung** (mặc định) để các container giao tiếp bằng tên service. Ví dụ: Airflow kết nối PostgreSQL bằng hostname `postgres` (nếu service đặt tên đó).
- Sử dụng **volumes** để duy trì dữ liệu:
 - Volume cho PostgreSQL data (`pgdata:/var/lib/postgresql/data`) để không mất dữ liệu Companies qua restart.
 - Volume cho HDFS namenode/datanode (nếu image hỗ trợ) để giữ dữ liệu thô. Nếu phức tạp, có thể chấp nhận HDFS ephemeral và mount thư mục host cho input/output. Dữ liệu HDFS cũng có thể được xuất sang host để backup.
 - Volume cho Airflow (dags, logs) để thuận tiện phát triển DAG (sửa file DAG trên host, container tự thấy).
 - Volume cho DuckDB file: một volume hoặc bind mount thư mục để chứa file `warehouse.duckdb`. Airflow or Spark container khi chạy sẽ read/write file này trên volume chung.

- Các biến môi trường quan trọng (API keys, database password) được quản lý trong file `.env` của compose và gán vào từng service. Điều này giữ cho thông tin nhạy cảm không bị viết thẳng trong file compose.

• Quy trình triển khai:

- Cài đặt Docker và Docker Compose trên máy chủ (có thể là máy local hoặc VM trên cloud như GCP Compute Engine đã dùng ở giai đoạn trước).
- Định nghĩa `docker-compose.yml` theo như trên.
- Chạy `docker-compose up -d` để khởi động tất cả services. Kiểm tra các container lên thành công.
- Vào container Airflow, load các **connection** (có thể qua UI hoặc CLI) để cấu hình kết nối Postgres, HDFS nếu cần. Tải lên DAG files (qua volume hoặc UI).
- Trigger DAG để chạy thử toàn bộ pipeline. Theo dõi logs trong Airflow UI, và kiểm tra kết quả: bảng trong DuckDB file đã có dữ liệu, Postgres có data, HDFS có file...
- Khi mọi thứ ổn định, Airflow scheduler sẽ duy trì chạy theo lịch, không cần can thiệp thêm.

• Ưu điểm của triển khai Docker Compose:

- Dễ dàng dựng môi trường tương tự trên bất kỳ máy nào chỉ với Docker (phù hợp cho các thành viên dev khác hoặc triển khai sau này).
- Cô lập các thành phần: ví dụ nâng cấp PostgreSQL hay Spark chỉ ảnh hưởng container đó, ít rủi ro cho toàn hệ thống.
- Quản lý cấu hình tập trung: file compose liệt kê rõ phiên bản dịch vụ, ports, env... giúp tài liệu hóa hệ thống.
- Nếu cần mở rộng sang nhiều máy (swarm hoặc k8s), có thể dùng file compose làm cơ sở.

(So với cách truyền thống cài từng thứ trên VM (như giai đoạn trước đã làm trên Google Compute Engine cài Hadoop, Spark thủ công), việc chuyển sang Docker Compose đơn giản hóa rất nhiều việc cài đặt/cấu hình và tăng tính reproducibility.)

Hướng Phát Triển Mở Rộng

Tài liệu trên mô tả phiên bản **batch processing** của hệ thống (dữ liệu được thu thập và xử lý theo lô tại các khoảng thời gian cố định). Trong tương lai, dự án có thể mở rộng thêm các hướng sau:

- **Tích hợp Streaming & Kafka:** Để xử lý **dữ liệu thời gian thực**, có thể bổ sung một kênh streaming. Ví dụ: sử dụng **Apache Kafka** để liên tục nhận các bản tin mới thay vì đợi theo lô. Mỗi khi có tin từ API (hoặc nguồn tin nhanh), đẩy vào Kafka Topic, và sử dụng **Spark Streaming** hoặc **Flink** xử lý trực tuyến, cập nhật ngay vào hệ thống. Tương tự, dòng giá cổ phiếu cũng có thể đưa vào Kafka (nếu có nguồn realtime) để phân tích tức thì. Việc này đòi hỏi triển khai thêm các thành phần Kafka, nhưng sẽ giúp hệ thống chuyển dần từ batch sang **lambda/streaming architecture**, cung cấp insight gần với thời gian thực hơn.
- **Mở rộng nguồn dữ liệu:** Bổ sung các nguồn khác như **dữ liệu mạng xã hội (Twitter)** về cổ phiếu, hoặc **dữ liệu kinh tế** (lãi suất, CPI từ API khác) để làm phong phú phân tích. Mỗi nguồn mới sẽ thêm task ETL tương ứng và mở rộng schema (ví dụ thêm fact SocialMediaSentiment, dim EconomicIndicators...).

- **Tối ưu và nâng cấp hạ tầng:** Khi dữ liệu tăng lên (vượt hàng trăm triệu bản ghi), xem xét triển khai **cluster** thực sự cho Hadoop/Spark (nhiều node), chuyển Data Warehouse từ DuckDB sang một hệ quản trị mạnh hơn (ví dụ: Snowflake, BigQuery hoặc SQL Data Warehouse), và container orchestration nâng cao (Kubernetes). Hiện tại DuckDB đủ cho quy mô nhỏ, nhưng tương lai cần mở rộng cho nhiều người dùng truy cập hoặc dữ liệu lớn hơn.

(Những hướng mở rộng này hiện chưa được thực hiện trong Project 1.6, nhưng nằm trong tầm nhìn dài hạn của kiến trúc hệ thống. Phiên bản hiện tại tập trung vào batch ETL nền tảng, sẵn sàng để tích hợp các thành phần nâng cao khi cần.)
